Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 8

Disciplina: Tehnici de programare

Tema: ” Lucrarea A. GitHub ”

A efectuat : st. gr.SI – 261 Talchiv Corina

A verificat : asistent universitar Dumitrascu Marius

Chișinău – 2026

SARCINA LUCRĂRII

CE FACEȚI ASTĂZI. Învățați GitHub de la zero, pas cu pas. Fiecare pas are comanda exactă și ce trebuie să apară pe ecran. La final aveți două depozite publice de exercițiu și știți să salvați, să trimiteți, să aduceți înapoi și să citiți istoria unui proiect. Lucrarea B, în fișa ei separată, continuă în același depozit.

CUM LUCRAȚI LA LUCRAREA A. Faceți pașii în ordine, fără să săriți. După fiecare comandă, comparați ce vedeți cu caseta verde „Pe ecran trebuie să vezi". Dacă nu seamănă, vă opriți și citiți tabelul de erori de la A12 sau chemați asistentul. Nu treceți mai departe cu un pas care nu a ieșit.

LUCRAREA A (nucleu). GitHub, pas cu pas
Obiectiv: un cont GitHub, git configurat pe calculatorul tău și două depozite publice de exercițiu, exercitiu-git și exercitiu-2, cu mai multe commit-uri trimise pe GitHub. În cuvinte simple: git ține istoria proiectului pe calculatorul tău, iar GitHub ține o copie a ei pe internet.

Cuvântul	Ce înseamnă
depozit (repository)	folderul proiectului, împreună cu toată istoria lui
commit	o fotografie a proiectului la un moment dat, cu un mesaj
push	trimiți commit-urile tale de pe calculator pe GitHub
pull	aduci de pe GitHub pe calculator ce s-a schimbat acolo
clone	descarci prima dată un depozit de pe GitHub
origin	numele scurt al copiei de pe GitHub

PAȘI REALIZABILE A LABORATORULUI:

A1. Contul GitHub

Dacă ai deja cont GitHub, autentifică-te și sari la A2.
1. Deschide în browser github.com și apasă Sign up.
2. Scrie adresa de e-mail pe care o citești zilnic (poate fi cea de student).
3. Alege o parolă și un nume de utilizator (username). Numele apare în toate linkurile tale: alege ceva profesional, de exemplu ion-popescu sau ipopescu, nu xX_hacker_Xx.
4. Rezolvă verificarea și confirmă codul primit pe e-mail.
5. Notează-ți numele de utilizator: îl folosești imediat.

De observat. GitHub îți poate cere mai târziu autentificarea în doi pași (2FA). Activeaz-o când ți-o cere, cu o aplicație de autentificare pe telefon: e același principiu pe care îl recomanzi oricui ca specialist în securitate.

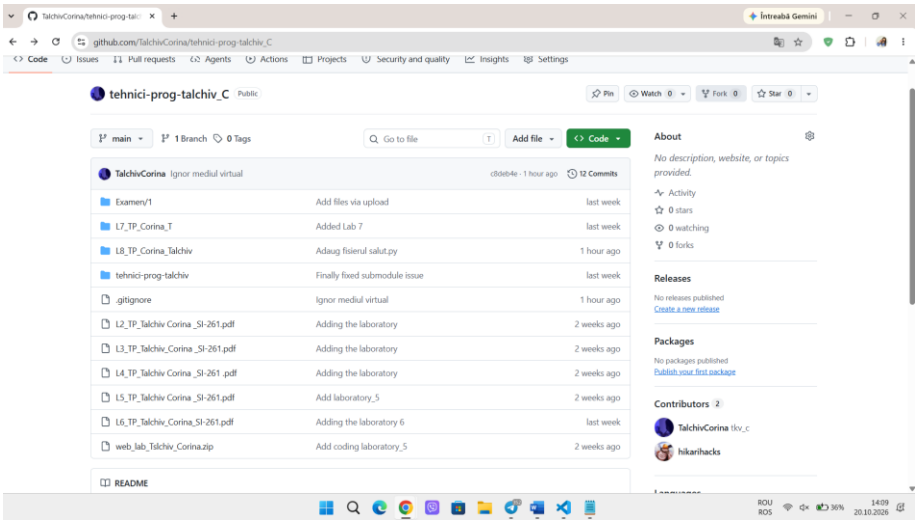


Figura 1

A2. Git pe calculator: verifici și te prezinți

Deschide terminalul (PowerShell pe Windows sau terminalul pe Linux) și verifică dacă git e instalat.

```
git --version
```

Pe ecran trebuie să vezi:

```
git version 2.53.0.windows.1      (numărul poate fi altul, nu contează)
```

Dacă vezi „git is not recognized" sau „command not found", git nu e instalat: pe Windows rulează winget install Git.Git, închide și redeschide terminalul; pe Linux, sudo apt install git.

Acum spui o singură dată cine ești. Pune numele tău real și EXACT adresa de e-mail din contul GitHub: așa GitHub leagă commit-urile de contul tău.

```
git config --global user.name "Ion Popescu"
git config --global user.email "adresa-ta@exemplu.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
```

Verifici ce ai setat:

```
git config --global --list
```

Pe ecran trebuie să vezi:

```
user.name=Ion Popescu
user.email=adresa-ta@exemplu.com
init.defaultbranch=main
pull.rebase=false
```

Ce fac ultimele două linii. `init.defaultBranch main` numește ramura principală „main", ca pe GitHub. `pull.rebase false` îi spune lui git cum să unească schimbările la pull, ca să nu te întrebe mai târziu.

```
(.venv) PS D:\Securitatea_Informationala_UTM\Tehnici_de_Programare\tehnici-prog-talchiv_C\exercitiu-2> git config --global --list
user.name=tkv_c
user.email=corinatalchiv@gmail.com
(.venv) PS D:\Securitatea_Informationala_UTM\Tehnici_de_Programare\tehnici-prog-talchiv_C\exercitiu-2>
```

Figura 2 - git config --global --list

A3. Primul depozit, creat pe GitHub (8 min)

În browser, pe GitHub:

1. Sus în dreapta apasă semnul + și alege New repository.
2. La Repository name scrie: exercitiu-git
3. La Description (opțional): Exercitii de git pentru Tehnici de programare
4. Alege Public.
5. Bifează Add a README file.
6. Apasă Create repository.

Ești acum pe pagina depozitului, la adresa https://github.com/<numele-tău>/exercitiu-git, și vezi fișierul README.md.

La acest pas deja am creat repositori-ul și adăugat fișierele

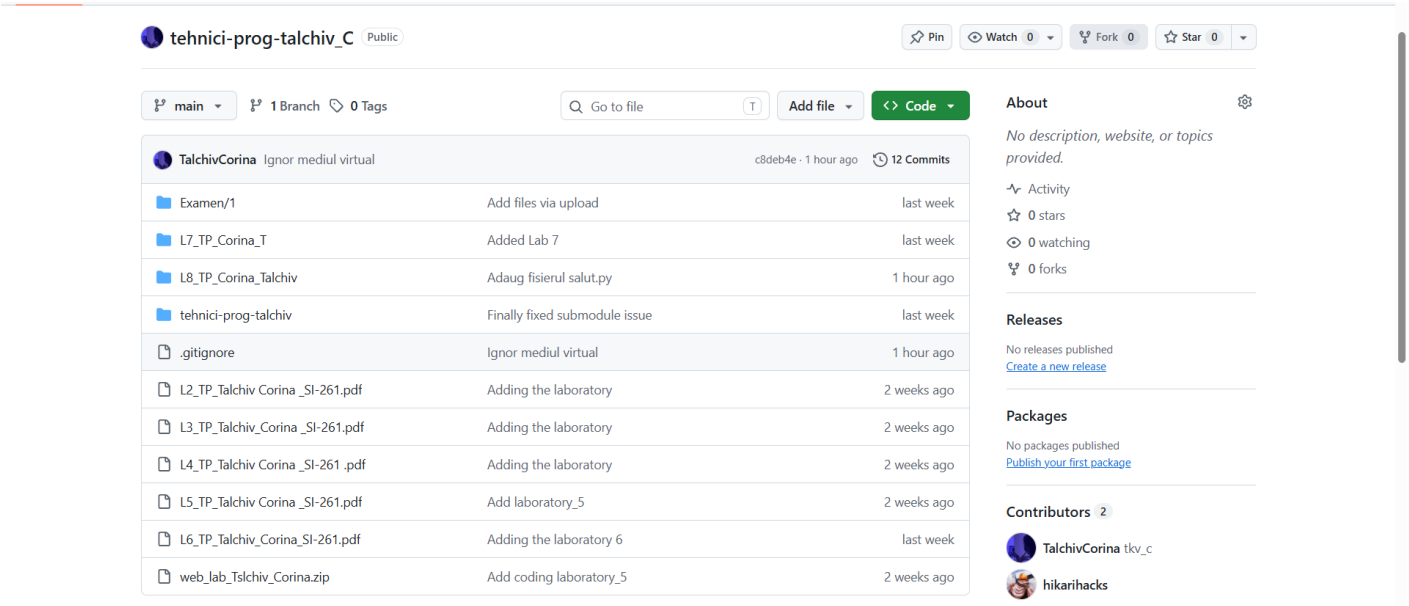


Figura 3 – repozitori-ul creat cu succes

A4. Îl aduci pe calculator: git clone (8 min)

Pe pagina depozitului apasă butonul verde Code, rămâi pe fila HTTPS și copiază adresa (se termină în .git). Apoi, în terminal, mergi într-un folder de lucru și clonează:

```
cd ~
git clone https://github.com/<numele-tau>/exercitiu-git.git
cd exercitiu-git
dir          # Linux: ls
```

Pe ecran trebuie să vezi:

```
Cloning into 'exercitiu-git'...
...
README.md
```

De observat. De acum lucrezi ÎN folderul exercitiu-git. Toate comenzile git de mai jos se dau de aici; dacă ieși din folder, git nu mai știe despre ce depozit vorbești.

Aici l-am clonat la mine local tor repozitori-ul

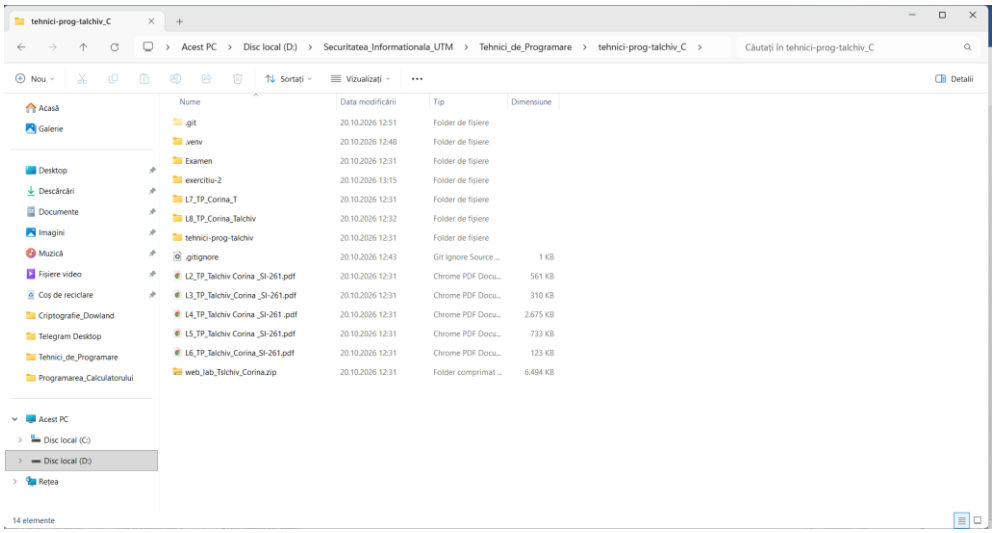


Figura 4 – Clone repository

A5. Prima ta fotografie: status, add, commit (12 min)

Creează un fișier nou, salut.py, din editor (nu cu echo din PowerShell, care scrie fișierul într-o codare pe care Python nu o citește). Scrie în el un singur rând, print("Salut, GitHub!"), salvează cu Ctrl+S, apoi rulează-l:

```
code salut.py        # sau deschizi folderul in editorul tau
python salut.py
```

Pe ecran trebuie să vezi:

```
Salut, GitHub!
```

Întreabă git ce vede:

```
git status
```

Pe ecran trebuie să vezi:

```
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
```

```
        salut.py
```

„Untracked" înseamnă: git vede fișierul, dar încă nu îl urmărește. Îl pui în fotografie cu add, apoi întrebi din nou:

```
git add salut.py
git status
```

Pe ecran trebuie să vezi:

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   salut.py
```

Fișierul e pregătit. Acum faci fotografia, cu un mesaj care spune ce ai schimbat:

```
git commit -m "Adaug salut.py"
```

Pe ecran trebuie să vezi:

```
[main aaf9ebc] Adaug salut.py
 1 file changed, 1 insertion(+)
 create mode 100644 salut.py
```

De observat. aaf9ebc e codul fotografiei; la tine e altul. Un avertisment de tipul „LF will be replaced by CRLF" e normal pe Windows și nu e o eroare.

```
(.venv) PS D:\Securitatea_Informationala_UTM\Tehnici_de_Programare\tehnici-prog-talchiv_C\L8_TP_Corina_Talchiv> python salut.py
Hello, GitHub!
(.venv) PS D:\Securitatea_Informationala_UTM\Tehnici_de_Programare\tehnici-prog-talchiv_C\L8_TP_Corina_Talchiv> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   salut.py
```

Figura 5 – Fișier creat salut.py

A6. O trimiți pe GitHub: git push (12 min)

```
git push
```

Prima dată, git îți cere să te autentifici la GitHub:

Pe Windows: se deschide o fereastră „Connect to GitHub". Alege Sign in with your browser, autentifică-te în browser și apasă Authorize. Fereastra se închide singură, iar push-ul continuă. Data viitoare nu te mai întreabă.

Pe Linux: când îți cere Username, scrii numele de utilizator GitHub. Când îți cere Password, NU merge parola contului: îți trebuie un token. Pe GitHub: poza ta (dreapta sus) → Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token (classic), bifezi repo, Generate, copiezi tokenul și îl lipești la Password. Păstrează-l ca pe o parolă.

Pe ecran trebuie să vezi:

```
To https://github.com/<numele-tau>/exercitiu-git.git
   e75c0d3..aaf9ebc  main -> main
```

Reîncarcă pagina depozitului în browser: apare salut.py, iar lângă el mesajul „Adaug salut.py".

```
(.venv) PS D:\Securitatea_Informationala_UTM\Tehnici_de_Programare\tehnici-prog-talchiv_C\L8_TP_Corina_Talchiv> git commit  -m "Adaug modificarea fisierului"
[main 9f1f19f] Adaug modificarea fisierului
 1 file changed, 1 insertion(+), 1 deletion(-)
(.venv) PS D:\Securitatea_Informationala_UTM\Tehnici_de_Programare\tehnici-prog-talchiv_C\L8_TP_Corina_Talchiv> git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 341 bytes | 341.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/TalchivCorina/tehnici-prog-talchiv_C
   c8deb4e..9f1f19f  main -> main
```

A8. Ce NU urcă pe GitHub: .gitignore (8 min)

Mediul virtual (.venv) are mii de fișiere care nu au ce căuta în depozit. Creează-l și vezi ce crede git:

```
python -m venv .venv
git status --short
```

Pe ecran trebuie să vezi:

```
?? .venv/
```

Dacă nu apare nimic, Python-ul tău (3.13 sau mai nou) a pus singur o regulă în .venv; faci oricum pasul, fiindcă .gitignore protejează și alte fișiere. Creează din editor un fișier numit exact .gitignore, cu două rânduri, și întreabă din nou:

```
# continutul fisierului .gitignore:
.venv/
__pycache__/

git status --short
```

Pe ecran trebuie să vezi:

```
?? .gitignore
```

.venv a dispărut din listă: git îl ignoră. Salvezi și trimiți regula:

```
git add .gitignore
git commit -m "Ignor mediul virtual"
git push
```

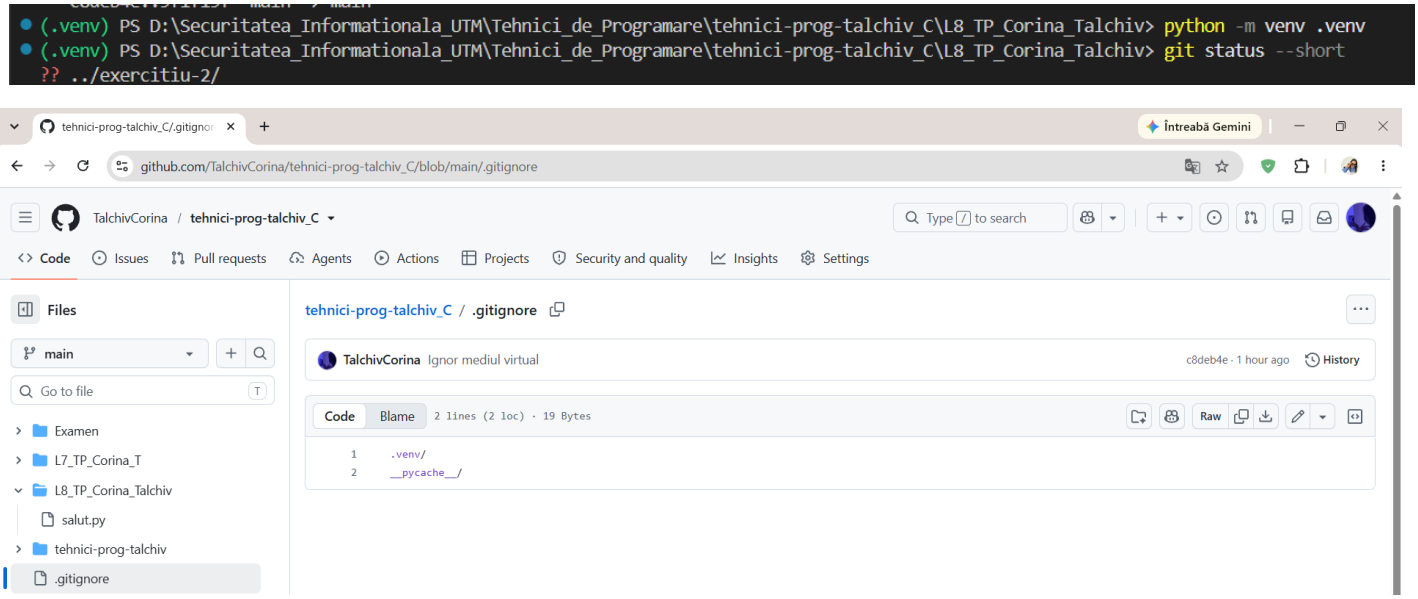


Figura 6 – Am adăugat .gitignore

Citești istoria și schimbările: log și diff (8 min)

```
git log --oneline
```

Pe ecran trebuie să vezi:

```
4f5a6b7 Ignor mediul virtual
1c2d3e4 Update README.md
aaf9ebc Adaug salut.py
e75c0d3 Initial commit
```

Adaugă un al doilea rând în salut.py, de exemplu print("al doilea"), și întreabă ce s-a schimbat, ÎNAINTE de commit:

```
git diff
```

Pe ecran trebuie să vezi:

```
--- a/salut.py
+++ b/salut.py
@@ -1 +1,2 @@
 print("Salut, GitHub!")
+print("al doilea")
```

Rândul cu + e adăugat, un rând cu - ar fi fost șters. Te-ai răzgândit? Întorci fișierul la ultima fotografie:

```
git restore salut.py
git status
```

Pe ecran trebuie să vezi:

```
nothing to commit, working tree clean
```

```
(.venv) PS D:\Securitatea_Informationala_UTM\Tehnici_de_Programare\tehnici-prog-talchiv_C\L8_TP_Corina_Talchiv> git log --oneline
9f1f19f (HEAD -> main, origin/main, origin/HEAD) Adaug modificarea fisierului
c8deb4e Ignor mediul virtual
140474b Adaug fisierul salut.py
18f40ef Add files via upload
e2fd910 Added Lab 7
b02e24e Finally fixed submodule issue
fdaa44b Fixing git submodule issue
b0a5304 Added lab 6
2b16174 Adding the laboratory 6
4d979eb Adding the laboratory 6
d3fa297 Add coding laboratory_5
a7d216e Add laboratory_5
1f5ddbd Adding the laboratory
```

Figura 7 – Comanda : git log --oneline

A10. Al doilea depozit, pornit de pe calculator (10 min)

De data asta pornești invers: întâi folderul local, apoi depozitul de pe GitHub. Ieși din exercitiu-git și creează un folder nou:

```
cd ~
mkdir exercitiu-2
cd exercitiu-2
git init
code README.md       # scrii un rand: # Al doilea depozit, si salvezi
git add README.md
git commit -m "Primul commit"
```

Pe GitHub creează un depozit nou exercitiu-2, Public, dar de data asta NU bifa Add a README file. GitHub îți arată o pagină cu instrucțiuni; folosești secțiunea „push an existing repository":

```
git remote add origin https://github.com/<numele-tau>/exercitiu-2.git
git branch -M main
git push -u origin main
```

Pe ecran trebuie să vezi:

```
branch 'main' set up to track 'origin/main'.
```

De observat. `-u` leagă ramura ta de cea de pe GitHub o singură dată; de acum încolo îți ajunge `git push` simplu.

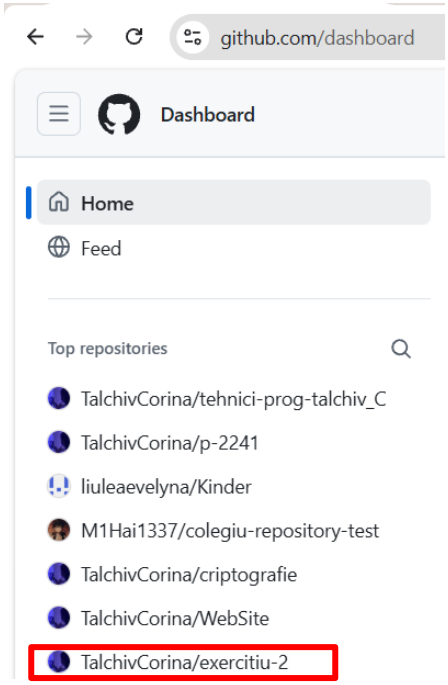


Figura 8 – Creez repository 2

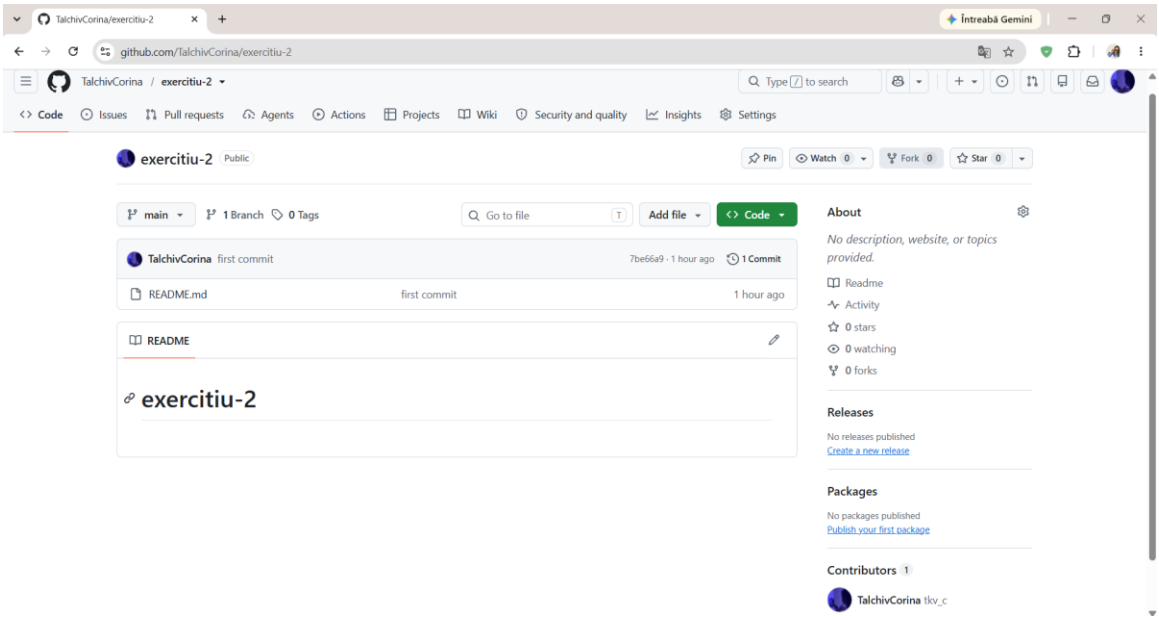


Figura 9 – Ce se află în repository

A11. Verificarea finală

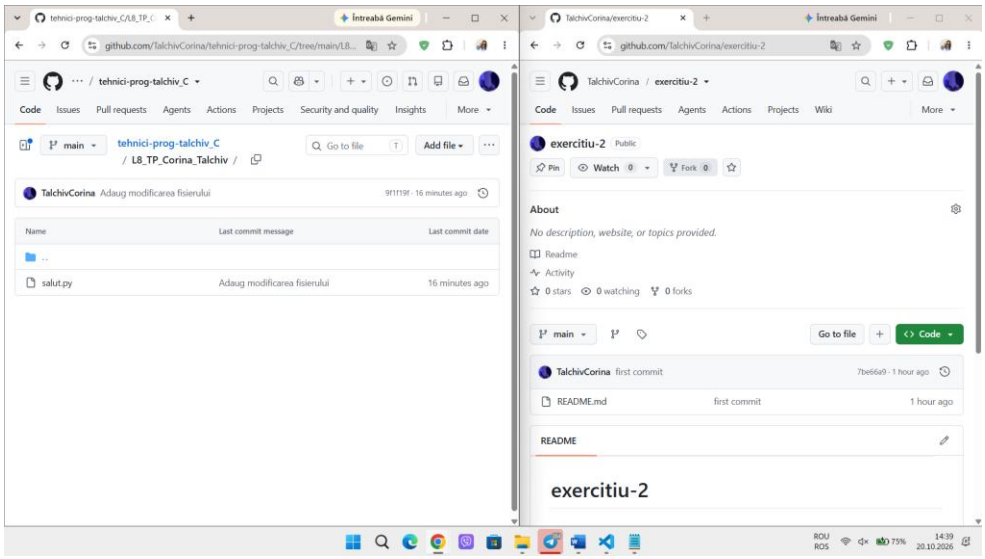


Figura 10 – Rezultatele finale

CONCLUZII

În cadrul acestui laborator, am parcurs pașii necesari pentru crearea, inițierea și configurarea unui repository local cu git init, gestionarea fișierelor de cod precum salut.py și sincronizarea modificărilor prin commit-uri succesive către platforma GitHub. Astfel, am consolidat bune practici în utilizarea controlului versiunilor pentru organizarea eficientă a codului sursă și urmărirea istoricului proiectului.

BIBLIOGRAFIE

1. https://github.com/TalchivCorina/tehnici-prog-talchiv_C/tree/main/L8_TP_Corina_Talchiv
2. https://github.com/TalchivCorina/exercitiu-2